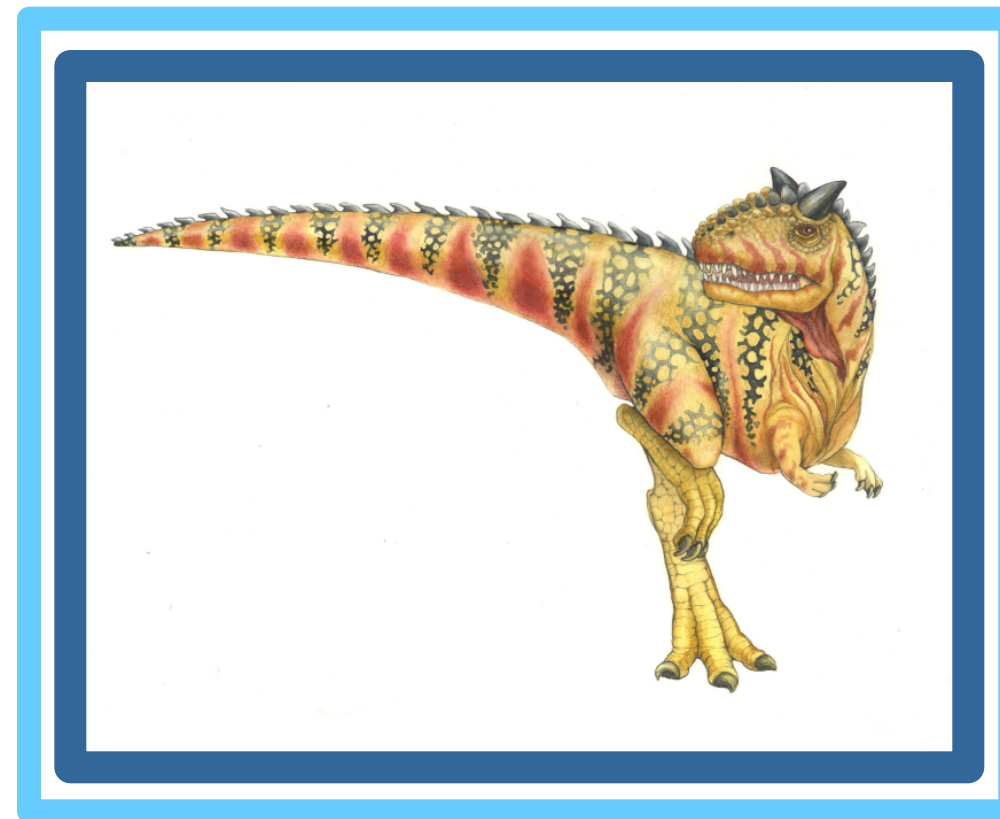
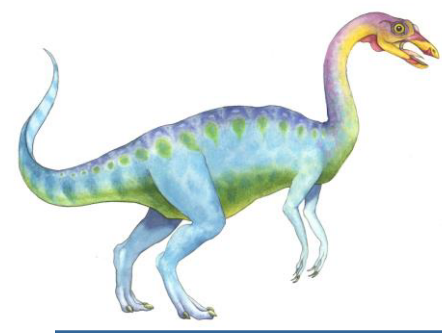


Chapter 8: Deadlocks





Chapter 8: Deadlocks

- ☐ System Model
- ☐ Deadlock Characterization
- ☐ Methods for Handling Deadlocks
- ☐ Deadlock Prevention
- ☐ Deadlock Avoidance
- ☐ Deadlock Detection
- ☐ Recovery from Deadlock





Methods for Handling Deadlocks

- Ensure that the system will *never* enter a deadlock state:
 - Deadlock prevention
 - Deadlock avoidance
- Allow the system to enter a deadlock state and then recover
- Ignore the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX

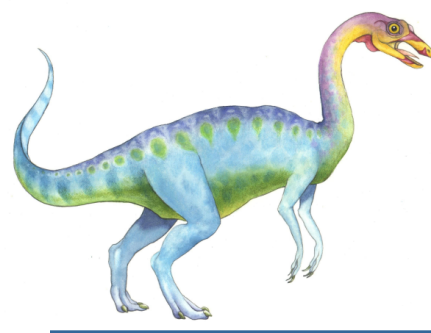




Avoidance Algorithms

- Single instance of a resource type
 - Use a resource-allocation graph
- Multiple instances of a resource type
 - Use the banker's algorithm





Banker's Algorithm

- Multiple instances
- We need prior information to avoid deadlock
- What kind of information we are looking for:
 - What is the max number of instances of a resource can a process request?
 - How many instances of a resource each process currently hold?
 - How many instances are available in the system?
- When a process requests a resource, it may have to wait
- When a process gets all its resources it must return them in a finite amount of time





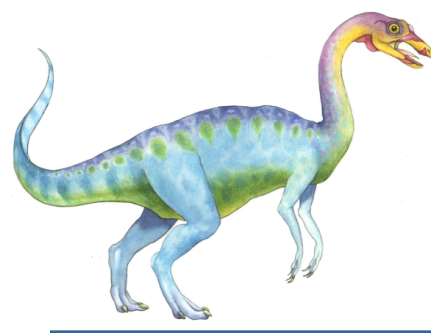
Example: P_1 Request (1,1,0,0)

- Need matrix?
- Safe sequence? Total instances for each resource type $\rightarrow A=12$ $B=12$ $C=8$ $D=10$

	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	2	0	0	1	4	2	1	2	3	3	2	1
P1	3	1	2	1	5	2	5	2				
P2	2	1	0	3	2	3	1	6				
P3	1	3	1	2	1	4	2	4				
P4	1	4	3	2	3	6	6	5				

- Safe sequence $\rightarrow P0$ $P3$ $P4$ $P1$ $P2$





Resource-Request Algorithm for Process P_i

$Request_i$ = request vector for process P_i . If **$Request_i[j] = k$** then process P_i wants k instances of resource type R_j

1. If **$Request_i \leq Need_i$** go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If **$Request_i \leq Available$** , go to step 3. Otherwise P_i must wait, since resources are not available
3. Pretend to allocate requested resources to P_i by modifying the state as follows:

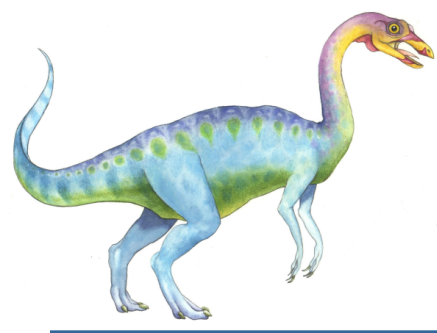
$Available = Available - Request_i;$

$Allocation_i = Allocation_i + Request_i;$

$Need_i = Need_i - Request_i;$

- If safe $\Rightarrow$ the resources are allocated to P_i
- If unsafe $\Rightarrow P_i$ must wait, and the old resource-allocation state is restored





- Need matrix?
- Safe sequence? Total instances for each resource type $\rightarrow A=12 \ B=12 \ C=8 \ D=10$

	Allocation				Max				Available				Need			
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D
P0	2	0	0	1	4	2	1	2	3	3	2	1	2	2	1	1
P1	3	1	2	1	5	2	5	2					2	1	3	1
P2	2	1	0	3	2	3	1	6					0	2	1	3
P3	1	3	1	2	1	4	2	4					0	1	1	2
P4	1	4	3	2	3	6	6	5					2	2	3	3

- Safe sequence $\rightarrow P0 \ P3 \ P4 \ P1 \ P2$





Example: P_1 Request (1,1,0,0)

- What is request from P_1 with (1,1,0,0), can request immediately granted?
- Check if request of $P_1 \leq$ need matrix $\rightarrow (1,1,0,0) \leq (2,1,3,1) \rightarrow \text{true}$
- Check if Request $\leq$ Available (that is, $(1,1,0,0) \leq (3,3,2,1) \rightarrow \text{true}$
- Update the need matrix, allocation matrix and available matrix.
- Now apply Bankers' algorithm.

	Allocation				Max				Available				Need			
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D
P0	2	0	0	1	4	2	1	2	2	2	2	1	2	2	1	1
P1	4	2	2	1	5	2	5	2					1	0	3	1
P2	2	1	0	3	2	3	1	6					0	2	1	3
P3	1	3	1	2	1	4	2	4					0	1	1	2
P4	1	4	3	2	3	6	6	5					2	2	3	3

- Now check if there is a safe sequence?
- Safe seq = P0 P3 P4 P1 P2
- As there is a safe seq, the system will accept the request of P_1





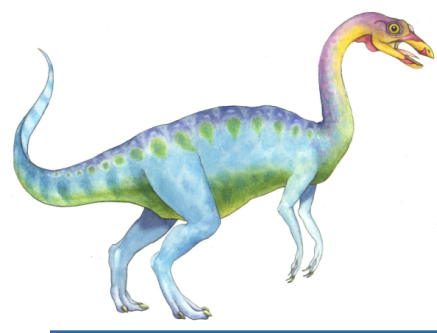
Example: P_4 Request (0,0,2,0)

- Check if p_4 request is $\leq$ the one in need matrix
- Check if p_4 request is $\leq$ available matrix
- Allocation matrix for p_4 will be updated (request + allocation), available matrix will be updated (available - request) and Need matrix will be updated (Need - request)
- Apply banker's algorithm to see if you can find a safe sequence

	Allocation				Max				Available				Need			
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D
P0	2	0	0	1	4	2	1	2	3	3	0	1	2	2	1	1
P1	3	1	2	1	5	2	5	2					2	1	3	1
P2	2	1	0	3	2	3	1	6					0	2	1	3
P3	1	3	1	2	1	4	2	4					0	1	1	2
P4	1	4	5	2	3	6	6	5					2	2	1	3

- Safe sequence??
- No safe sequence so the request won't be accepted.





Example: P_4 Request (0,0,1,0)

- Can request for (0, 0, 1, 0) by P_4 be granted?

	Allocation				Max				Available				Need			
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D
P0	2	0	0	1	4	2	1	2	3	3	0	1	2	2	1	1
P1	3	1	2	1	5	2	5	2					2	1	3	1
P2	2	1	0	3	2	3	1	6					0	2	1	3
P3	1	3	1	2	1	4	2	4					0	1	1	2
P4	1	4	5	2	3	6	6	5					2	2	1	3





Avoidance Algorithms

- Single instance of a resource type
 - Use a resource-allocation graph
- Multiple instances of a resource type
 - Use the banker's algorithm





Chapter 8: Deadlocks

System Model
Deadlock Characterization
Methods for Handling Deadlocks
Deadlock Prevention
Deadlock Avoidance
Deadlock Detection
Recovery from Deadlock

